Материалът за десетата лекция от записките:
глава 3.8 (**Алгоритми**), страници 127-133, 135-136.

Нека $G = (V, \Sigma, R, S)$ е безконтекстна граматика.

В предната лекция доказахме лемата за покачване за безконтекстни езици, като избрахме числото $p = b^{|V|+1}$, където b е максималната дължина на дясна част на правило от R. Както при регулярни езици, съществуването на числото p дава метод за проверка за празнота, както и за безкрайност на езика на безконтекстна граматика:

$$L(G) \neq \emptyset \iff \exists \alpha \in L(G)\ |\alpha| < p,$$

$$|L(G)| = \infty \iff \exists \alpha \in L(G)\ p \leq |\alpha| < 2p.$$

Доказателството е аналогично на това при регулярни езици. Разбира се това показва, че проблемите за празнота и за безкрайност се свеждат до проблема за принадлежност: при дадени безконтекстна G и $\alpha \in \Sigma^\star$ дали $\alpha \in L(G)$? За този проблем ще видим ефективен алгоритъм в края на лекцията.

Премахване на безполезните променливи

Една променлива нарекохме *полезна*, ако участва съществено в извеждането на някоя дума от $\Sigma^\star$ в граматиката. По-точно, $A \in V$ е полезна, ако съществува извод $S \overset{\star}{\Rightarrow} \alpha A \beta \overset{\star}{\Rightarrow} \gamma$, където $\alpha, \beta \in (V \cup \Sigma)^\star$ и $\gamma \in \Sigma^\star$.

Ясно е, че безполезните променливи могат да бъдат премахнати от граматиката, заедно с правилата в които участват, без това да се отрази на извеждания език.

За да открием полезните променливи разбиваме условието в дефиницията на две части: една променлива A е *достижима*, ако $S \overset{\star}{\Rightarrow} \alpha A \beta$ за $\alpha, \beta \in (V \cup \Sigma)^\star$ и *генерираща*, ако $A \overset{\star}{\Rightarrow} \gamma$ за $\gamma \in \Sigma^\star$.

Възможно е в една безконтекстна граматика променлива да е достижима и генерираща без да е полезна (помислете за пример). От друга страна съобразете, че ако *всяка* променлива е достижима и генерираща, то *всяка* променлива е полезна.

Твърдение 1 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ с $L(G) \neq \emptyset$ съществува безконтекстна граматика $G' = (V', \Sigma, R', S)$, такава че $L(G') = L(G)$ и в която всяка променлива е генерираща, т.е. за всяка променлива $A \in V'$ съществува $\gamma \in \Sigma^\star$, такава че $A \overset{\star}{\Rightarrow}_{G'} \gamma$.*

Доказателство. Нека

$$Gen = \{A \in V \mid \exists \gamma \in \Sigma^\star\ A \overset{\star}{\lhd} \gamma\}.$$

Това е множеството от генериращите променливи в G. Ще го пресметнем на стъпки като дефинираме:

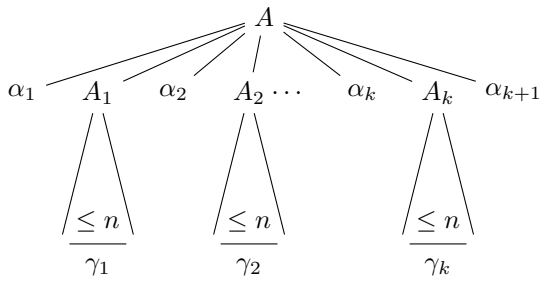
$$Gen[n] \ = \ \{A \in V \mid \exists \gamma \in \Sigma^\star\ A \overset{\leq n}{\lhd} \gamma\}.$$

Разбира се, $Gen[0] = \emptyset$ и също

$$Gen[n+1] \;=\; \{A \in V \mid \exists \alpha \in (Gen[n] \cup \Sigma)^\star \; A \to_G \alpha\}.$$

С други думи, една променлива A попада в $Gen[n+1]$, т.е. има дърво на извод с височина $\leq n+1$ и корен A извеждащо дума от $\Sigma^\star$, точно когато има правило в граматиката $A \to_G \alpha$, чиято дясна част α съдържа букви от Σ и променливи само от $Gen[n]$.

Ситуацията може да се илюстрира по следния начин:
($\alpha = \alpha_1 A_1 \alpha_2 A_2 \ldots \alpha_k A_k \alpha_{k+1}$ и $\alpha_i \in \Sigma^\star$, $\gamma_i \in \Sigma^\star$)



Разбира се, има по едно ребро от A към всяка буква в думите α_i. Лесно се съобразява, че:

$$Gen[0] \;\subseteq\; Gen[1] \;\subseteq\; Gen[2] \;\subseteq\; \ldots \;\subseteq\; Gen[n] \;\subseteq\; Gen[n+1] \;\subseteq\; \ldots.$$

Тъй като V е крайно множество, тази верига се стабилизира, т.е. ще дойде момент n, такъв че $Gen[n] = Gen[n+1]$. Тогава множеството Gen, което търсим съвпада с последното $Gen[n]$.

За да образуваме новата граматика G' трябва да дефинираме $V' = Gen$ и $R' = R \cap (V' \times (V' \cup \Sigma)^\star)$. В изводите от вида $S \stackrel{\star}{\Rightarrow}_G \gamma$ за $\gamma \in \Sigma^\star$ участват само променливи от Gen и от това следва, че $L(G) = L(G')$.

Следствие 2 *$L(G) = \emptyset$ точно когато $S \notin Gen$.*

Този метод за проверка за празнота е значително по-ефективен от метода, който описахме по-горе.

Твърдение 3 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ съществува безконтекстна граматика $G' = (V', \Sigma, R', S)$, такава че $L(G') = L(G)$ и всяка променлива в G' е достижима, т.е. за всяка $A \in V'$ съществуват $\alpha, \beta \in (V' \cup \Sigma)^\star$, такива че $S \stackrel{\star}{\Rightarrow}_{G'} \alpha A \beta$.*

Доказателство. Нека

$$Reach = \{A \in V \mid \exists \alpha, \beta \in (V \cup \Sigma)^\star \; S \stackrel{\star}{\lhd} \alpha A \beta\}.$$

Това е множеството от достижимите променливи в G. Както в предното твърдение, пресмятаме го на стъпки:

$$Reach[n] = \{A \in V \mid \exists \alpha, \beta \in (V \cup \Sigma)^\star \; S \overset{\leq n}{\lhd} \alpha A \beta\}.$$

Започваме с $Reach[0] = \{S\}$ (тривиалното дърво с корен S) и след това

$$Reach[n{+}1] \; = \; Reach[n] \cup \{A \in V \mid \exists B \in Reach[n] \; B \to_G \alpha A \beta \text{ за } \alpha, \beta \in (V \cup \Sigma)^\star\}.$$

По този начин, за да е достижима променливата A на стъпка $n+1$ (но не на стъпка n) трябва да има достижима променлива B на стъпка $\leq n$ и също да има правило от вида $B \to_G \alpha A \beta$. Както по-горе, спираме на първото n, такова че $Reach[n] = Reach[n+1]$ и търсеният резултат $Reach$ съвпада с това последно $Reach[n]$. Забележете, че за разлика от предходното твърдение тук по дефиниция добавяме $Reach[n]$ към $Reach[n+1]$ за да постигнем растяща верига относно включването. В алгоритъма за Gen това се получава автоматично, тъй като $Gen[0] = \emptyset$. Но в случая $Reach[0] = \{S\}$ и това само по себе си не гарантира, че S ще попадне в $Reach[n]$ за $n \geq 1$.

Новата граматика G' се образува като се вземе $V' = Reach$ и правила $R' = R \cap (V' \times (V' \cup \Sigma)^\star)$. В изводите с начало S очевидно участват само достижими променливи и затова лесно се вижда, че $L(G) = L(G')$.

Пример. Нека е дадена следната граматика:

$$S \to AB \mid ab, \; A \to aA \mid \epsilon, \; B \to bB.$$

Изпълняваме първо алгоритъма за Gen: $Gen[0] = \emptyset$, $Gen[1] = \{S, A\}$ заради правилата $S \to ab$ и $A \to \epsilon$, $Gen[2] = \{S, A\}$ и алгоритъмът приключва, т.е. $Gen = \{S, A\}$. Променливата B не влиза на стъпка 2, тъй като няма правило с лява част B и дясна част от $(\Sigma \cup \{S, A\})^\star$. Новата граматика G' се образува като запазим само правилата, в които участват S и A:

$$S \to ab, \; A \to aA \mid \epsilon.$$

После изпълняваме алгоритъма за $Reach$: $Reach[0] = \{S\}$ и $Reach[1] = \{S\} = Reach$, тъй като няма правила с лява част S, които съдържат променливи в дясната част. В новата граматика G'' запазваме правилата, в които участва само S: $S \to ab$.

Забележете, че редът на прилагане на Gen и $Reach$ е съществен. Да започнем първо с $Reach$. Имаме $Reach[0] = \{S\}$, след това $Reach[1] = \{S, A, B\}$ заради правилото $S \to AB$ и разбира се $Reach = Reach[1] = \{S, A, B\}$, тъй като това са всички променливи. Естествено, трябва да запазим и всички правила. След това прилагаме Gen и резултатът е както горния, т.е. новата граматика има правилата $S \to ab, \; A \to aA \mid \epsilon$ и в нея A е безполезна променлива, тъй като не е достижима.

Този пример показва, че след изпълняване на Gen една достижима променлива може да стане недостижима. От друга страна, след изпълнение

на *Reach* не е възможно генерираща променлива да стане негенерираща, тъй като недостижимост на коя да е променлива в извода $A \stackrel{\star}{\Rightarrow} \gamma$ влече недостижимост и на A.

Твърдение 4 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ съществува безконтекстна граматика $G' = (V', \Sigma, R', S)$, такава че $L(G') = L(G)$ и всяка променлива в G' е полезна. Също така, G' може да се построи ефективно от G.*

Доказателство. При дадена G прилагаме първо *Gen* и след това *Reach*.

Премахване на ϵ-правила

Правилата от вида $A \to \epsilon$ наричаме *ϵ-правила*. Следващото твърдение показва, че можем да ги елиминираме от граматиката, с цената на това, че трябва да премахнем празната дума от езика.

Твърдение 5 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ съществува безконтекстна граматика $G' = (V, \Sigma, R', S)$, в която няма ϵ-правила и също така $L(G') = L(G) \setminus \{\epsilon\}$.*

Доказателство. Първо ще си поставим за цел да генерираме всички променливи A, които извеждат ϵ. Нека

$$E = \{A \in V \mid A \stackrel{\star}{\lhd} \epsilon\}.$$

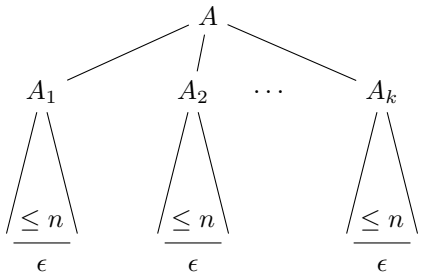
Отново ще пресмятаме E на стъпки:

$$E[n] \;=\; \{A \in V \mid A \stackrel{\leq n}{\lhd} \epsilon\}.$$

В началото $E[0] = \emptyset$ и след това

$$E[n+1] \;=\; \{A \in V \mid \exists \alpha \in E[n]^\star \; A \to_G \alpha\}.$$

Ситуацията е много подобна на алгоритъма за *Gen*: $(\alpha = A_1 A_2 \ldots A_k)$



Разбира се, букви от Σ не могат да участват в изводи на празната дума. Да напомним, че $\emptyset^\star = \{\epsilon\}$ и затова $E[1] = \{A \in V \mid A \to_G \epsilon\}$, т.е. на стъпка 1 трябва да вземем променливите от левите части на ϵ-правилата в граматиката. Отново ще настъпи момент n, в който $E[n] = E[n+1]$ и търсеното множество E съвпада с последното $E[n]$.

В новата граматика G' трябва да включим следните правила:

$$R' = \{A \to Y_1 Y_2 \dots Y_k \mid \exists A \to_G X_1 X_2 \dots X_k,\ Y_i = X_i \text{ или } (Y_i = \epsilon \ \&\ X_i \in E)\}.$$

Разбира се, поне едно Y_i трябва да е различно от ϵ. В най-лошия случай, $X_i \in E$ за всяко i и така заради правилото $A \to_G X_1 X_2 \dots X_k$ трябва да добавим $2^k - 1$ правила в R'. Това показва, че алгоритъмът в общия случай е експоненциален. Не е трудно да се съобрази, че $L(G') = L(G) \setminus \{epsilon\}$, тъй като дърветата на извод съвместими с G' се получават от дърветата на извод съвместими G с премахване на всички поддървета, които извеждат празната дума.

Пример. Да предположим, че $B, D \in E,\ C \notin E$.

Тогава на правилото $A \to aBCDb$ в G съответстват 4 правила в G': $A \to aBCDB \mid aCDb \mid aBCb \mid aCb$.

На правилото $A \to BD$ в G съответстват 3 правила в G': $A \to BD \mid B \mid D$.

Премахване на преименуващите правила

Правилата от вида $A \to B$, където $A, B \in V$ наричаме *преименуващи*. Нашата цел ще е да ги елиминираме от граматиката.

Твърдение 6 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ в която няма ϵ-правила, съществува безконтекстна граматика $G' = (V, \Sigma, R', S)$, в която няма преименуващи правила и $L(G') = L(G)$.*

Доказателство. Ще генерираме всички двойки от променливи, такива че $A \stackrel{\star}{\Rightarrow}_G B$. Забележете, че в G няма ϵ-правила и затова всеки извод $A \stackrel{\star}{\Rightarrow}_G B$ всъщност има вида $A \Rightarrow_G C_1 \Rightarrow_G C_2 \Rightarrow_G \dots \Rightarrow_G C_n \Rightarrow_G B$. При наличието на ϵ-правила в такъв извод може да участва и дума с дължина повече от 1 (например, $A \Rightarrow_G BC \Rightarrow_G B$, ако има правило $C \to_G \epsilon$).

Дефинираме множеството

$$Ren = \{(A, B) \in V \times V \mid A \stackrel{\star}{\Rightarrow}_G B\}$$

и го пресмятаме на стъпки:

$$Ren[n] = \{(A, B) \in V \times V \mid A \stackrel{\leq n}{\Rightarrow}_G B\}.$$

Тук няма особена разлика дали разглеждаме дърво на извод или линеен извод (изглеждат еднакво).

Започваме с $Ren[0] = \{(A, A) \mid A \in V\}$ и след това

$$Ren[n+1] = Ren[n] \cup \{(A, B) \mid \exists C \in V \ \ A \to_G C \ \&\ (C, B) \in Ren[n]\}.$$

Отново трябва да добавим $Ren[n]$, ако искаме растяща верига, тъй като $Ren[0]$ не се съдържа автоматично в $Ren[1]$. Спираме на първото n, такова че $Ren[n] = Ren[n+1]$ и резултатът Ren съвпада с последното $Ren[n]$.

Използвайки изучавани понятия можем да кажем, че Ren е рефлексивното и транзитивно затваряне на релацията $\{(A, B) \mid A \to_G B\}$. Също можем да си мислим, че тази релация представя ориентиран граф и тогава $(A, B) \in Ren$ точно когато има маршрут с начало A и край B.

Остава да покажем как се образуват правилата в новата граматика G':

$$R' = \{A \to \alpha \mid \exists B \in V \ (A, B) \in Ren \ \ \& \ \ B \to_G \alpha \ \ \& \ \ \alpha \notin V\}.$$

Разбира се, $\alpha \notin V$ означава, че в R' не допускаме преименуващи правила. Можем да обобщим идеята по следния начин. Бихме искали изводът

$$A \Rightarrow_G C_1 \Rightarrow_G C_2 \Rightarrow_G \ \ldots \ \Rightarrow_G C_n \Rightarrow_G B \Rightarrow_G \beta$$

в G да се симулира в новата граматика на една стъпка: $A \Rightarrow_{G'} \beta$ и затова добавяме правилото $A \to_{G'} \beta$ на основание на това, че $(A, B) \in Ren$, т.е. има преименуваща верига с начало A и край B и също $B \to_G \beta$ е (непреименуващо) правило в G.

Премахване на дългите правила

Правилата от вида $A \to \beta$, където $|\beta| \geq 3$ наричаме *дълги правила*.

Нека $A \to X_1 X_2 \ldots X_k$ е дълго правило, $k \geq 3$. Можем да го заменим със следните правила:

$$A \to X_1 A_1, \ \ A_1 \to X_2 A_2, \ \ \ldots, \ \ A_{k-2} \to X_{k-1} X_k,$$

където $A_1, A_2, \ldots, A_{k-2}$ са нови променливи. Разбира се, за различните дълги правила ще използваме различни нови променливи.

Нормална форма на Чомски

Безконтекстната граматика G е в *нормална форма на Чомски*, ако правилата в G са от вида $A \to BC$ или $A \to a$, където $A, B, C \in V$, $a \in \Sigma$.

Теорема 7 *За всеки безконтекстен език $L = L(G)$, такъв че $\epsilon \notin L$ съществува безконтекстна граматика G' в нормална форма на Чомски, такава че $L(G') = L$. При това, G' може да се построи ефективно от G.*

Доказателство. Започваме с безконтекстна граматика G и върху нея извършваме опростяванията в следния ред:

1. Премахваме дългите правила.
2. Премахваме ϵ-правилата.
3. Премахваме преименуващите правила.
4. Премахваме безполезните променливи (не е задължително).

Това че премахваме дългите правила преди алгоритъма за ϵ-правилата гарантира, че няма да се получи експоненциалното добавяне на нови правила, което споменахме по-горе. На всяко старо правило ще съответстват най-много 4 нови правила.

Съществено е да премахнем ϵ-правилата преди преименуващите правила, тъй като стана ясно, че използваният алгоритъм разчита на това.

След изпълняване на опростяванията получените правила имат следния вид: $A \to a,\ A \to BC,\ A \to bX,\ A \to Yc,\ A \to bc$.

За да премахнем последните три вида правила избираме по една нова променлива U за всяка буква $u \in \Sigma$ и добавяме правилото $U \to u$. След това заместваме правилата $A \to bX,\ A \to Yc,\ A \to bc$ съответно с $A \to BX,\ A \to YC,\ A \to BC$. Така получената граматика вече е в нормална форма на Чомски.

CYK алгоритъм

Дадена е $G = (V, \Sigma, R, S)$ в нормална форма на Чомски и дума $\alpha \in \Sigma^\star$, $\alpha = a_0 a_1 \dots a_{n-1}$. Въпросът, който искаме да разрешим е дали $\alpha \in L(G)$?

Идеята на CYK алгоритъма е да потърсим отговора на по-общия въпрос дали $A \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s}$ за $A \in V$ и $i \leq i+s < n$ и затова дефинираме

$$V[i][i+s] = \{A \in V \mid A \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s}\}.$$

При $s = 0$ получаваме $V[i][i] = \{A \in V \mid A \to_G a_i\}$ (няма друг начин да се изведе a_i, тъй като G е в нормална форма на Чомски).

При $s > 0$ основната идея се корени в следното наблюдение:

$$A \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s} \iff \exists B, C \in V \ \&\ k \in \mathbb{N},\ i \leq k < i+s,$$
$$A \to_G BC \ \&\ B \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_k \ \&\ C \stackrel{\star}{\Rightarrow}_G a_{k+1} \dots a_{i+s}.$$

То следва лесно от основното твърдение за безконтекстни граматики и от факта, че граматиката е в нормална форма на Чомски. По този начин, въпросът за думата $a_i a_{i+1} \dots a_{i+s}$ се свежда към същия въпрос за по-късите думи $a_i a_{i+1} \dots a_k$ и $a_{k+1} \dots a_{i+s}$. В това е същността на динамичното оптимиране. Започваме със $s = 0$ и постепенно пресмятаме множествата $V[i][i+s]$ за думи с все по-голяма дължина. Накрая отговорът на оригиналния въпрос е $\alpha \in L(G)$, ако $S \in V[0][n-1]$ и $\alpha \notin L(G)$, ако $S \notin V[0][n-1]$. Псевдокод на CYK алгоритъма може да видите на страница 135 в записките.